Multivariate Statistics Homework 1

Yiyang YU

Comments:

Utilized AI for plotting and implementing the Python equivalent of `factanal` since I ran into dependency issues with python factor_analyzer library

In [12]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from numpy.linalg import eig
from scipy.stats import norm, spearmanr, kendalltau
from IPython.display import display, Markdown
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

Problem 1: Descriptive multivariate analysis

In [52]:
```
import pandas as pd

df = pd.read_csv('ladyrun25.dat', sep='\t', names=['Country', '100m', '200m'
print(df.head())
df.columns
```

```
  Country   100m   200m   400m  800m  1500m  3000m  Marathon
0     ARG  11.17  22.94  52.50  2.05   4.20   9.19    150.32
1     AUS  11.11  22.23  48.63  1.97   4.01   8.63    142.60
2     AUT  11.15  22.70  50.62  1.94   4.05   8.78    154.35
3     BEL  11.04  22.20  51.45  1.97   4.08   8.82    143.05
4     BER  11.28  23.05  53.30  2.07   4.29   9.81    174.18
```

Out[52]:
```
Index(['Country', '100m', '200m', '400m', '800m', '1500m', '3000m',
       'Marathon'],
      dtype='str')
```

a. Sample Means

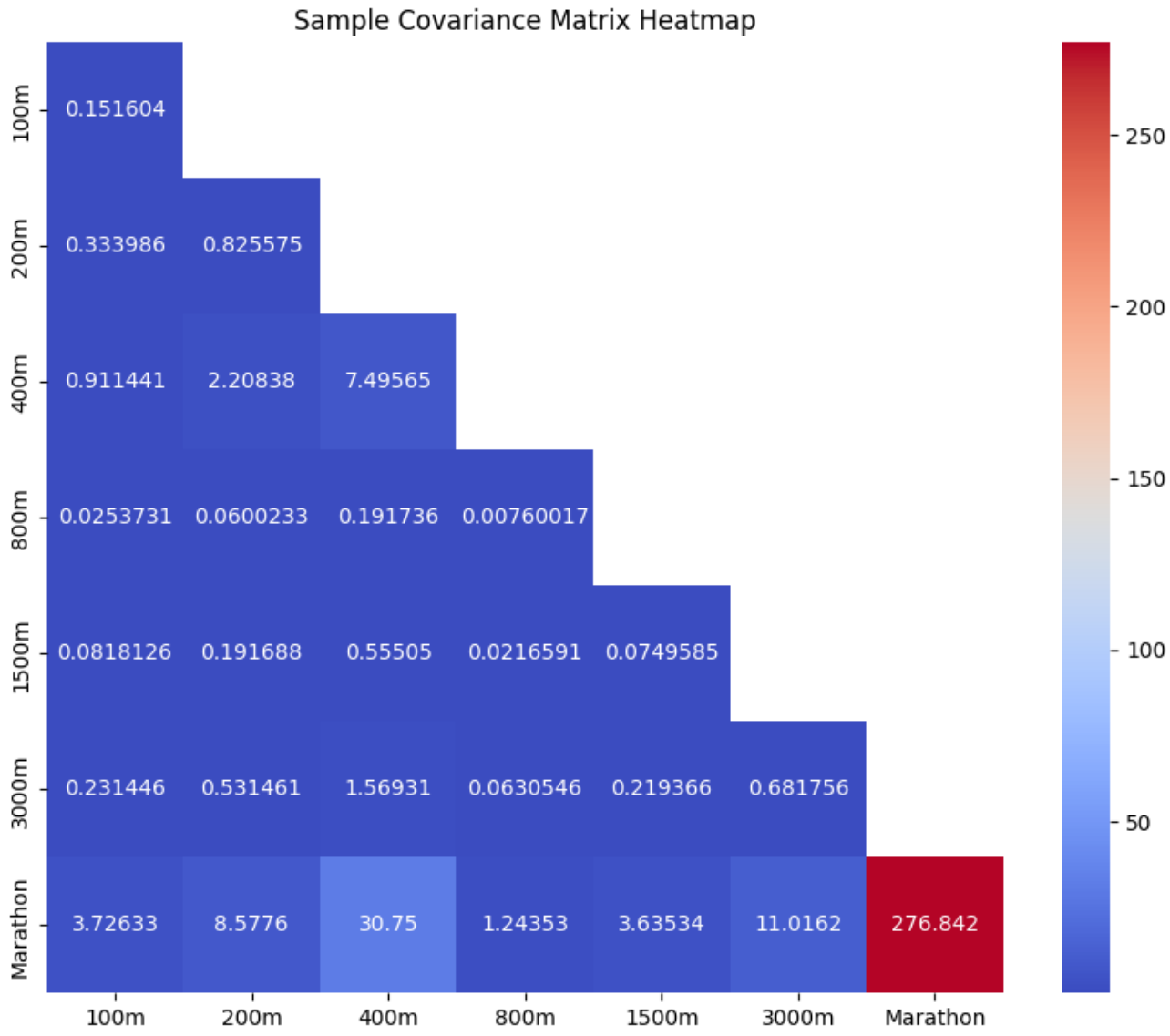
In [3]:
```
mean = df[['100m', '200m', '400m', '800m', '1500m', '3000m', 'Marathon']].me
std = df[['100m', '200m', '400m', '800m', '1500m', '3000m', 'Marathon']].std
display(mean)
display(std)
```

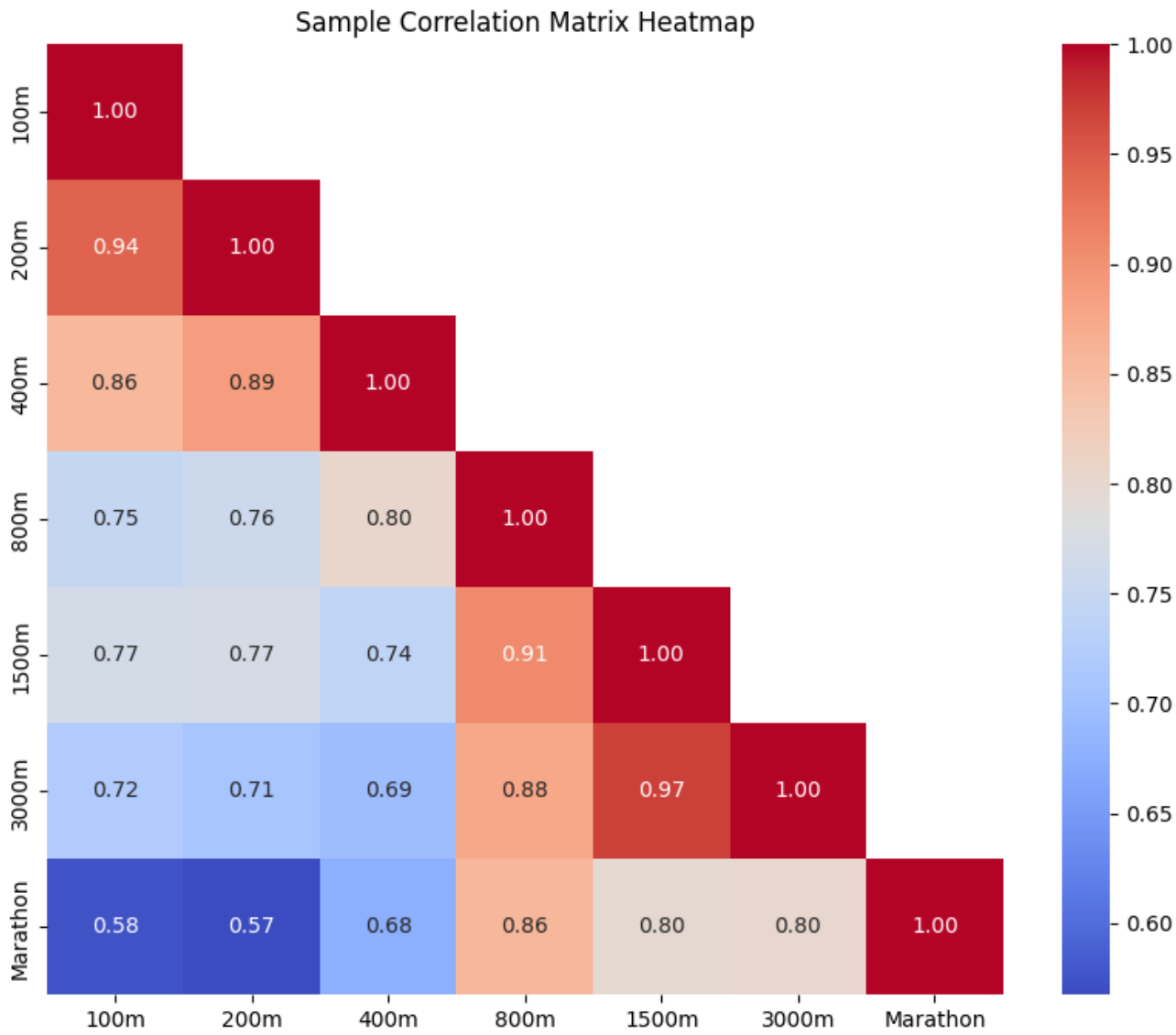
```
100m         11.310370
200m         23.070926
400m         51.820185
800m          2.021296
1500m         4.186667
3000m         9.067222
Marathon    153.306852
dtype: float64
100m         0.389363
200m         0.908611
400m         2.737819
800m         0.087179
1500m        0.273785
3000m        0.825685
Marathon    16.638562
dtype: float64
```

- The sample mean for the Marathon national track records is less interpretable than those of the other columns because of the high standard deviation and high mean since the marathon is measured in terms of minutes and the sample distribution of Marathon times is significantly higher than those of other variables.

b. Sample covariance matrix and sample correlation matrix

In [4]:
```
cov = np.cov(df[['100m', '200m', '400m', '800m', '1500m', '3000m', 'Marathor
corr = np.corrcoef(df[['100m', '200m', '400m', '800m', '1500m', '3000m', 'Ma

mask = np.triu(np.ones_like(cov, dtype=bool), k=1)

import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 8))

sns.heatmap(cov, annot=True, mask=mask, cmap='coolwarm', fmt='g', xticklabel
plt.title('Sample Covariance Matrix Heatmap')
plt.show()

plt.figure(figsize=(10, 8))

sns.heatmap(corr, annot=True, mask=mask, cmap='coolwarm', fmt='.2f', xtickla
plt.title('Sample Correlation Matrix Heatmap')
plt.show()
```

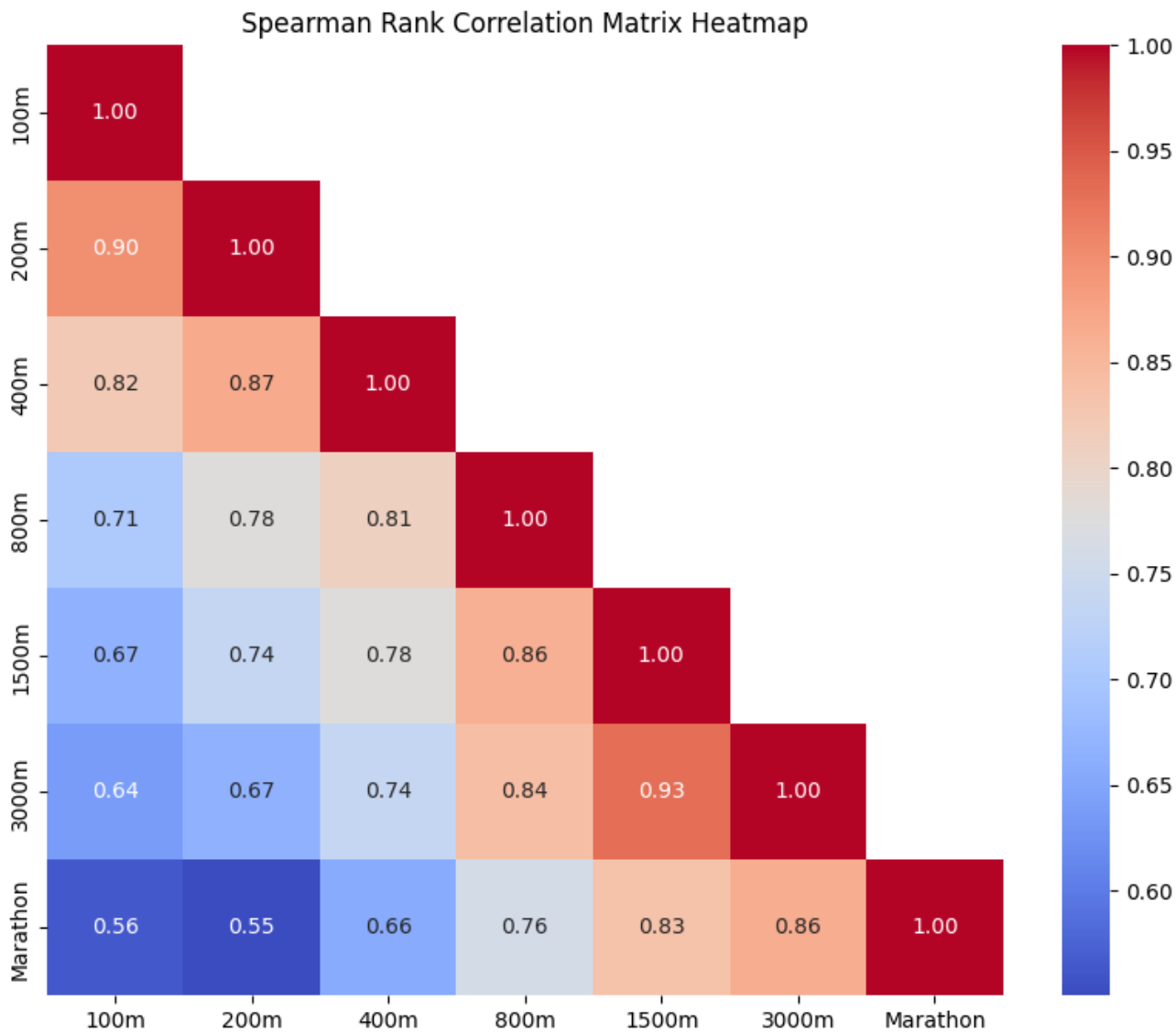

Sample Covariance Matrix Heatmap
100m
200m
400m
800m
1500m
3000m
Marathon
0.151604
0.333986 0.825575
0.911441 2.20838 7.49565
0.0253731 0.0600233 0.191736 0.00760017
0.0818126 0.191688 0.55505 0.0216591 0.0749585
0.231446 0.531461 1.56931 0.0630546 0.219366 0.681756
3.72633 8.5776 30.75 1.24353 3.63534 11.0162 276.842
250
200
150
100
50



c. Spearman's Rank and Kendall's tau

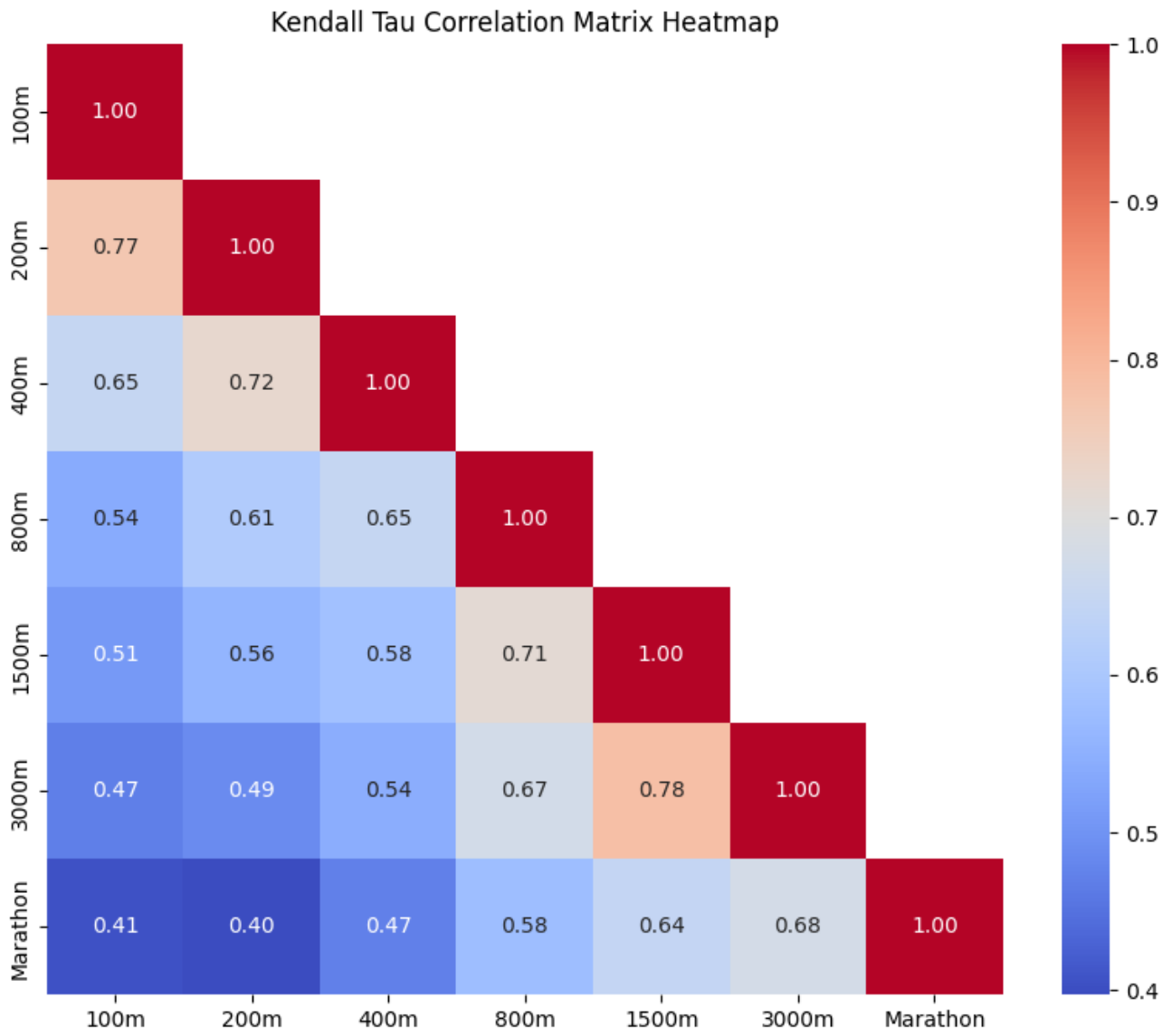
In [5]:
```
track_rank = spearmanr(df[['100m', '200m', '400m', '800m', '1500m', '3000m',
display(track_rank)
plt.figure(figsize=(10, 8))
sns.heatmap(track_rank, annot=True, cmap='coolwarm', fmt='.2f', mask = np.tr
plt.title('Spearman Rank Correlation Matrix Heatmap')
plt.show()
```

```
array([[1.        , 0.89904304, 0.82278602, 0.70504423, 0.66662856,
        0.63680729, 0.56415828],
       [0.89904304, 1.        , 0.86752316, 0.77846367, 0.73664744,
        0.66687001, 0.55047271],
       [0.82278602, 0.86752316, 1.        , 0.80965196, 0.77832865,
        0.73799585, 0.65624762],
       [0.70504423, 0.77846367, 0.80965196, 1.        , 0.85989858,
        0.84126611, 0.75892434],
       [0.66662856, 0.73664744, 0.77832865, 0.85989858, 1.        ,
        0.93070443, 0.83441064],
       [0.63680729, 0.66687001, 0.73799585, 0.84126611, 0.93070443,
        1.        , 0.86384691],
       [0.56415828, 0.55047271, 0.65624762, 0.75892434, 0.83441064,
        0.86384691, 1.        ]])
```



In [7]:
```
results_matrix = []

cols = df.columns[1:8]
```

```
for i in range(len(cols)):
    row = []
    for j in range(len(cols)):
        tau, _ = kendalltau(df[cols[i]].tolist(), df[cols[j]].tolist())
        row.append(tau)
    results_matrix.append(row)

kendall_array = np.array(results_matrix)

# 4. Plotting
plt.figure(figsize=(10, 8))
# Create the mask using the correct shape of the final array
mask = np.triu(np.ones_like(kendall_array, dtype=bool), k=1)

sns.heatmap(
    kendall_array,
    annot=True,
    cmap='coolwarm',
    fmt='.2f',
    mask=mask,
    xticklabels=cols,
    yticklabels=cols
)

plt.title('Kendall Tau Correlation Matrix Heatmap')
plt.show()
```



d.

- Based on the outputs above, we see that the Spearman and Pearsons' rau values are quite similar, with the kendall's tau having a generally smaller value, this can be found In the last row of the correlation matrix where we compare the correlations of other track times to the marathon times. We can conclude that although there is a monotonic relationship between track times, it is slightly non-linear.

e. Pearson eigvenvalues and eigenvectors

In [10]:
```
eigvenvalues, eigvectors = eig(corr)
print("Eigenvalues:", eigvenvalues)
print("Eigenvectors:\n", eigvectors)
```

```
Eigenvalues: [5.70130407 0.73690632 0.28859891 0.02195575 0.05074016 0.11440
672
 0.08608808]
Eigenvectors:
 [[-0.37203419  0.45751952 -0.14870245 -0.08348107  0.56774254  0.52629124
  -0.15450205]
 [-0.37387836  0.48015626 -0.07423786  0.20389904 -0.74952583  0.11131548
  -0.09164471]
 [-0.3747904   0.33148106  0.48724807 -0.0737348   0.19965203 -0.50849863
   0.45647911]
 [-0.39491234 -0.22107701  0.14789147  0.15592393  0.12121193 -0.37710528
  -0.76947015]
 [-0.39565823 -0.23057567 -0.42485979 -0.75036695 -0.14315474 -0.13992068
   0.08162078]
 [-0.38342889 -0.31807486 -0.47659266  0.59797109  0.14018729 -0.07501674
   0.37659087]
 [-0.34902548 -0.4970255   0.55267291 -0.03296996 -0.14553495  0.53351836
   0.13707747]]
```

f. Sum of eigenvalues

In [11]:
```
sum(eigvenvalues)
```

Out[11]:
```
np.float64(7.000000000000001)
```

- The sum of the eigenvalues is 7, which matches exactly with the number of variables in the data.

g. Interpretation

The first eigenvalue is much larger than the rest, meaning the first principal component alone explains most of the variation in the data. This makes sense since countries that are fast in short sprint events also tend to be fast in longer distance events, so all seven variables move together. The remaining eigenvalues are small, suggesting the additional principal components capture relatively minor variation beyond this dominant factor.

Problem 2: Covariance And Spectral Decomposition

a. Let Y $\in \mathbb{R}^p$ be a random vector with covaraince matrix $\sum$. Show that $\sum$ is positive semi-definite, all p eigenvalues of $\sum$ must be non-negative.

$$\sum = E[(Y-\mu)(Y-\mu)^T]$$
$$\text{Using the definition of positive semi-definess: } v^T \sum v \geq 0 \quad \forall v \in \mathbb{R}^p$$
$$v^T \sum v = v^T E[(Y-\mu)(Y-\mu)^T]v$$
$$= E[v^T(Y-\mu)(Y-\mu)^T v] = E[(v^T(Y-\mu))^2] \geq 0$$

- Since the term inside the expectation is squared, the covariance matrix is positive-semi-definite

b. Find the eigvenvalues and unit-length eigenvectors of A below

$$\begin{pmatrix} 1 & p \\ p & 1 \end{pmatrix}$$
$$p \in (0,1)$$

$$A - \lambda I = \begin{pmatrix} 1-\lambda & p \\ p & 1-\lambda \end{pmatrix}$$
$$det(A - \lambda I) = 0$$
$$(1-\lambda)^2 - p^2 = 0$$
$$\pm(1-\lambda) = p$$
$$\lambda = 1-p, \quad p+1$$

$$For \quad \lambda = 1-p$$
$$A - \lambda I = \begin{pmatrix} p & p \\ p & p \end{pmatrix} = 0$$
$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \rightarrow \frac{1}{\sqrt{(2)}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

$$For \quad \lambda = p+1$$
$$A - \lambda I = \begin{pmatrix} -p & p \\ p & -p \end{pmatrix} = 0$$
$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \rightarrow \frac{1}{\sqrt{(2)}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

c. Spectral Decomposition $A = V\Lambda V^T$

$$\begin{pmatrix} 1 & p \\ p & 1 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} 1-p & 0 \\ 0 & p+1 \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix}$$

d. A^{-1} in terms of matrix operations on V and A

Since $A = V\Lambda V^T$ and V is orthogonal ($V^{-1} = V^T$), we have:

$$A^{-1} = (V\Lambda V^T)^{-1} = V\Lambda^{-1}V^T$$

Substituting explicitly:

$$A^{-1} = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} \frac{1}{1-p} & 0 \\ 0 & \frac{1}{1+p} \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} = \frac{1}{1-p^2} \begin{pmatrix} 1 & -p \\ -p & 1 \end{pmatrix}$$

Problem 3: PCA analysis on running-record

a.

- Covariance PCA is preferred when the data has similar units/scale, versus correlation PCA is preferred when the units are mixed. Since the columns in the running-record dataset have different units (seconds, minutes, etc). A correlation PCA is more appropriate for this dataset due to the mixed scaling.

b.

In [19]:
```
numerical_df = df[['100m', '200m', '400m', '800m', '1500m', '3000m', 'Marath
scaler = StandardScaler()
X_scaled = scaler.fit_transform(numerical_df)
pca = PCA(n_components=2)
principal_components = pca.fit_transform(X_scaled)
loadings = pd.DataFrame(
    pca.components_.T,
    index=numerical_df.columns,
    columns=['PC1', 'PC2']
)
for pc in ['PC1', 'PC2']:
    terms = ' + '.join(f"({loadings[pc][col]:.3f})*{col}" for col in numeric
```

```
    print(f"{pc} = {terms}\n")
```

```
PC1 = (0.372)*100m + (0.374)*200m + (0.375)*400m + (0.395)*800m + (0.396)*15
00m + (0.383)*3000m + (0.349)*Marathon

PC2 = (-0.458)*100m + (-0.480)*200m + (-0.331)*400m + (0.221)*800m + (0.23
1)*1500m + (0.318)*3000m + (0.497)*Marathon
```

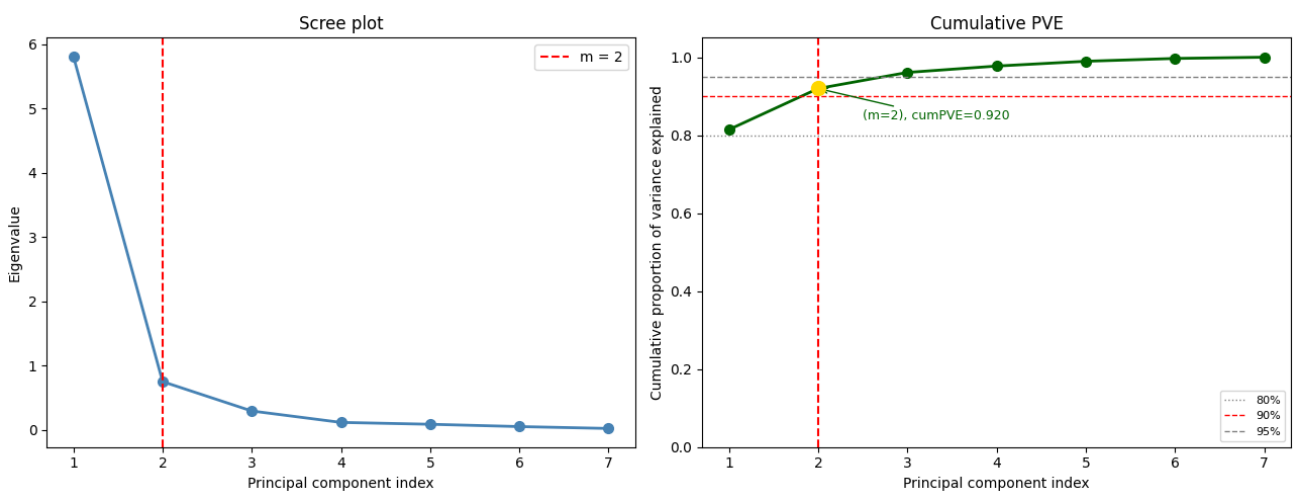
- Based on the result above, it is clear that the first two principal components can be expressed below:

$$PC1 = (0.372) \cdot 100m + (0.374) \cdot 200m + (0.375) \cdot 400m + (0.395) \cdot 800m + ($$
$$PC2 = (-0.458) \cdot 100m + (-0.480) \cdot 200m + (-0.331) \cdot 400m + (0.221) \cdot 800m + ($$

c. Scree Plot and Cumulative Proportion of Varaince Explained By the First Two Principal Components

In [23]:
```
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import numpy as np

pca_full = PCA(n_components=7)
pca_full.fit(X_scaled)

eigenvalues = pca_full.explained_variance_  # actual eigenvalues
ev = pca_full.explained_variance_ratio_
cumev = np.cumsum(ev)
n = len(eigenvalues)
x = np.arange(1, n + 1)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))

# --- Scree Plot ---
ax1.plot(x, eigenvalues, 'o-', color='steelblue', linewidth=2, markersize=7)
ax1.axvline(x=2, color='red', linestyle='--', linewidth=1.5, label='m = 2')
ax1.set_xlabel('Principal component index')
ax1.set_ylabel('Eigenvalue')
ax1.set_title('Scree plot')
ax1.set_xticks(x)
ax1.legend()

# --- Cumulative PVE Plot ---
ax2.plot(x, cumev, 'o-', color='darkgreen', linewidth=2, markersize=7)

# Threshold lines
ax2.axhline(y=0.80, color='gray', linestyle=':', linewidth=1, label='80%')
```

```
ax2.axhline(y=0.90, color='red', linestyle='--', linewidth=1, label='90%')
ax2.axhline(y=0.95, color='gray', linestyle='--', linewidth=1, label='95%')

# Selected m vertical line
selected_m = 2
ax2.axvline(x=selected_m, color='red', linestyle='--', linewidth=1.5)

# Annotate the selected point
ax2.annotate(f'(m={selected_m}), cumPVE={cumev[selected_m-1]:.3f}',
             xy=(selected_m, cumev[selected_m-1]),
             xytext=(selected_m + 0.5, cumev[selected_m-1] - 0.08),
             fontsize=9, color='darkgreen',
             arrowprops=dict(arrowstyle='->', color='darkgreen'))
ax2.plot(selected_m, cumev[selected_m-1], 'o', color='gold', markersize=10,

ax2.set_xlabel('Principal component index')
ax2.set_ylabel('Cumulative proportion of variance explained')
ax2.set_title('Cumulative PVE')
ax2.set_xticks(x)
ax2.set_ylim(0, 1.05)
ax2.yaxis.set_major_formatter(mtick.FuncFormatter(lambda v, _: f'{v:.1f}'))
ax2.legend(loc='lower right', fontsize=8)

plt.tight_layout()
plt.show()
```

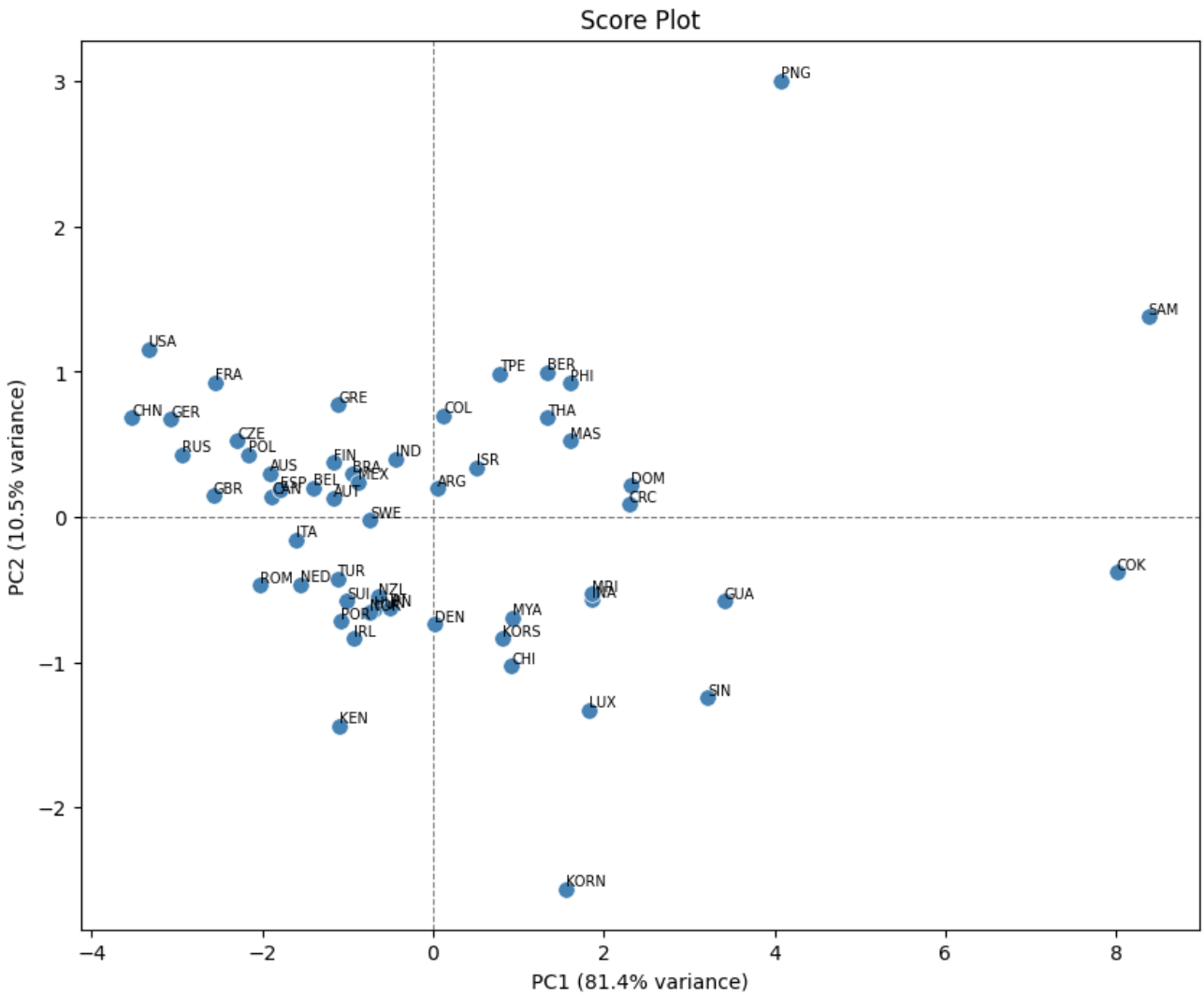


- Based on the plot above, I would choose the first two principal components to explain the data while maintaining low dimensional summary. From the Scree plot, we see that the plot flattens after the second principal component, with very small eigenvalues after the first two (< 0.5), and based on the cumulative PVE plot, the first two principal component captures explains around 92% of the total varaince.

d. Score Plot

In [27]:
```
scores = principal_components  # from your original pca.fit_transform
loadings = pca.components_.T   # shape (7, 2)
labels = numerical_df.columns

plt.figure(figsize=(10, 8))
plt.scatter(scores[:, 0], scores[:, 1], color='steelblue', edgecolors='white
            linewidth=0.5, s=70, zorder=3)

# Label each country if you have a country column
for i, country in enumerate(df['Country']):  # adjust column name as needed
    plt.annotate(country, (scores[i, 0], scores[i, 1]),
                 fontsize=7, ha='left', va='bottom')

plt.axhline(0, color='gray', linewidth=0.8, linestyle='--')
plt.axvline(0, color='gray', linewidth=0.8, linestyle='--')
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)')
plt.title('Score Plot')
```

Out[27]:
```
Text(0.5, 1.0, 'Score Plot')
```

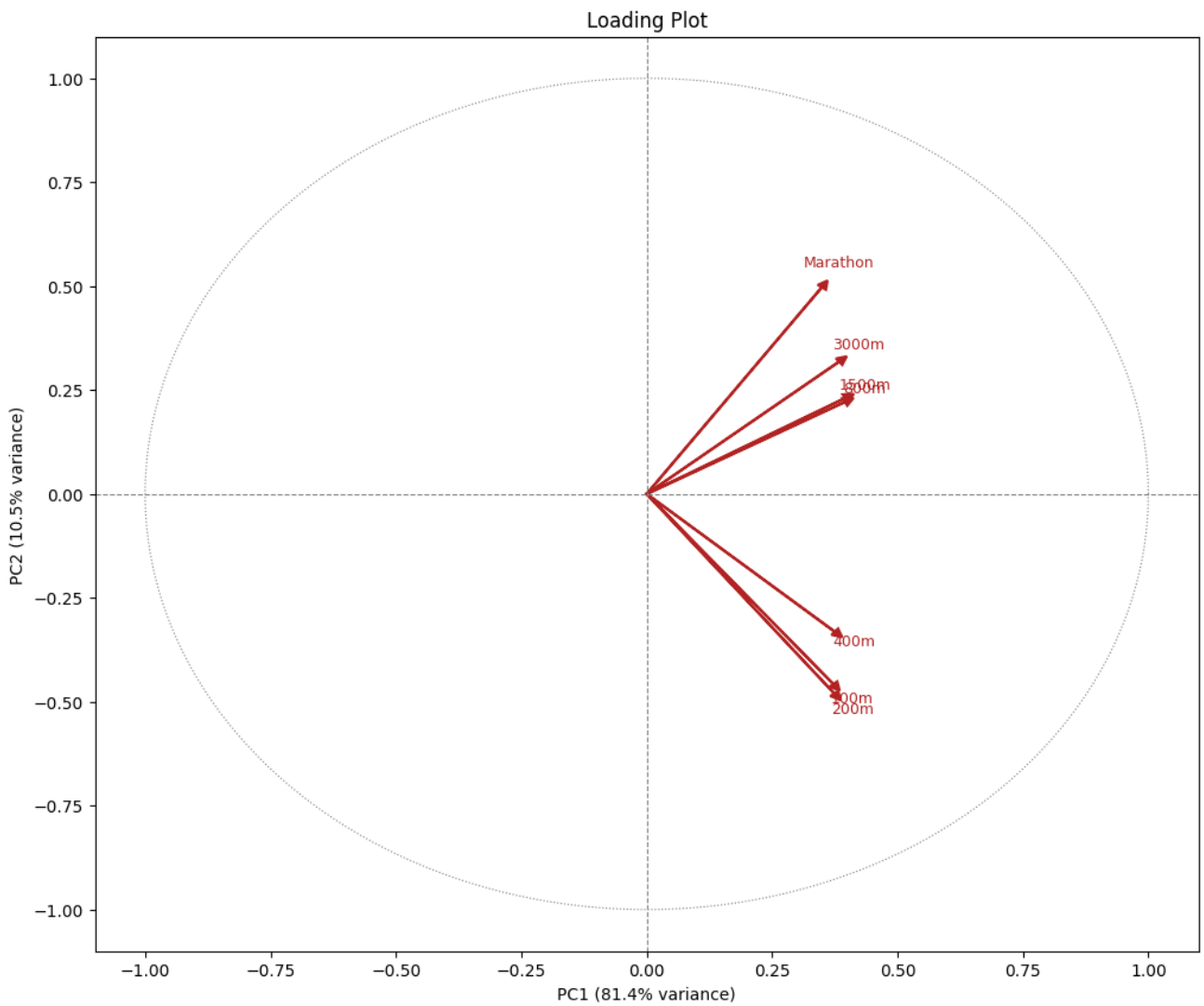


- The scores plot demonstrates the original data projected onto the two main principal components. The two principal components show no linear relationship, as expected by construction. It is interesting that based on this score, the performance of countries differ based on the region and their advantages. The PCs of european countries are quite similar, with USA and China leading on the left side of the PC1 axis. PC1 seems to capture the country's overall athletic ability.

e. Loadings Plot

In [31]:
```
plt.figure(figsize=(12, 10))

for i, var in enumerate(labels):
    plt.arrow(0, 0, loadings[i, 0], loadings[i, 1],
              head_width=0.02, head_length=0.02,
              fc='firebrick', ec='firebrick', linewidth=1.5)
```

```
    plt.annotate(var, (loadings[i, 0]*1.1, loadings[i, 1]*1.1),
                 fontsize=9, color='firebrick', ha='center')
plt.axhline(0, color='gray', linewidth=0.8, linestyle='--')
plt.axvline(0, color='gray', linewidth=0.8, linestyle='--')

# Unit circle for reference
theta = np.linspace(0, 2*np.pi, 200)
plt.plot(np.cos(theta), np.sin(theta), color='gray', linewidth=0.8, linestyl


plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)')
plt.title('Loading Plot')
plt.show()
```



- All variables load positively and roughly equally on PC1 (loadings around 40%), consistent with PC1 representing an overall athletic performance index, countries with faster times across all events score lower on PC1. PC2 differentiates event type:

sprint events (100m, 200m, 400m) load negatively while distance events (800m, 1500m, 3000m, Marathon) load positively, suggesting PC2 captures a country's relative specialization in short versus long distance running.

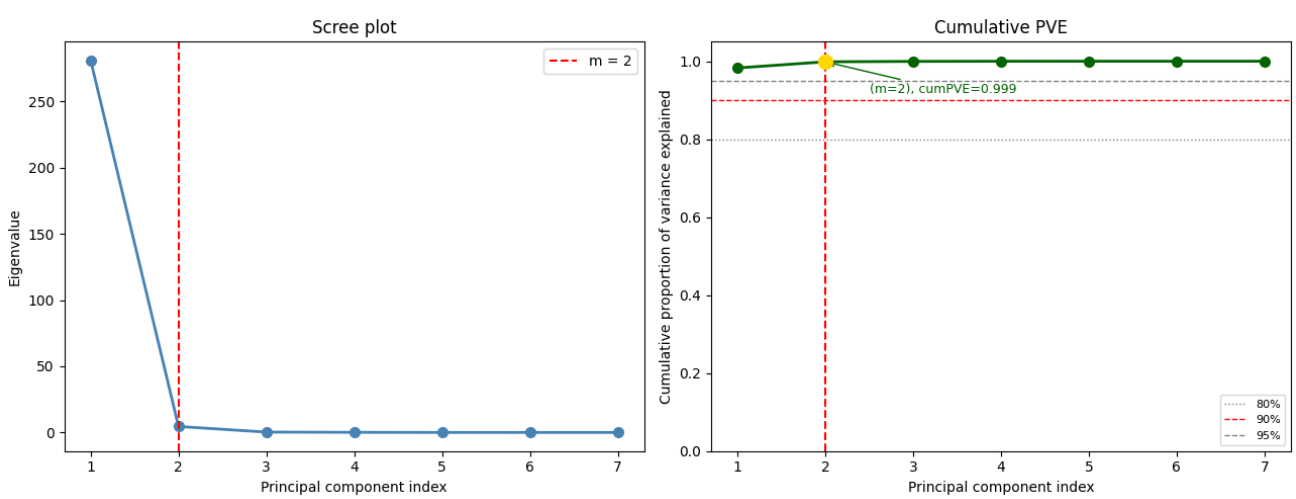
f. PCA with covariance matrix

In [34]:
```
scaler_cov = StandardScaler(with_std=False)
X_scaled_cov = scaler_cov.fit_transform(numerical_df)
pca_cov = PCA(n_components=2)
principal_components_cov = pca_cov.fit_transform(X_scaled_cov)
loadings_cov = pd.DataFrame(
    pca_cov.components_.T,
    index=numerical_df.columns,
    columns=['PC1', 'PC2']
)
for pc in ['PC1', 'PC2']:
    terms = ' + '.join(f"({loadings_cov[pc][col]:.3f})*{col}" for col in num
    print(f"{pc} = {terms}\n")
```

```
PC1 = (0.014)*100m + (0.031)*200m + (0.112)*400m + (0.004)*800m + (0.013)*15
00m + (0.040)*3000m + (0.992)*Marathon

PC2 = (0.120)*100m + (0.303)*200m + (0.933)*400m + (0.012)*800m + (0.037)*15
00m + (0.087)*3000m + (-0.121)*Marathon
```

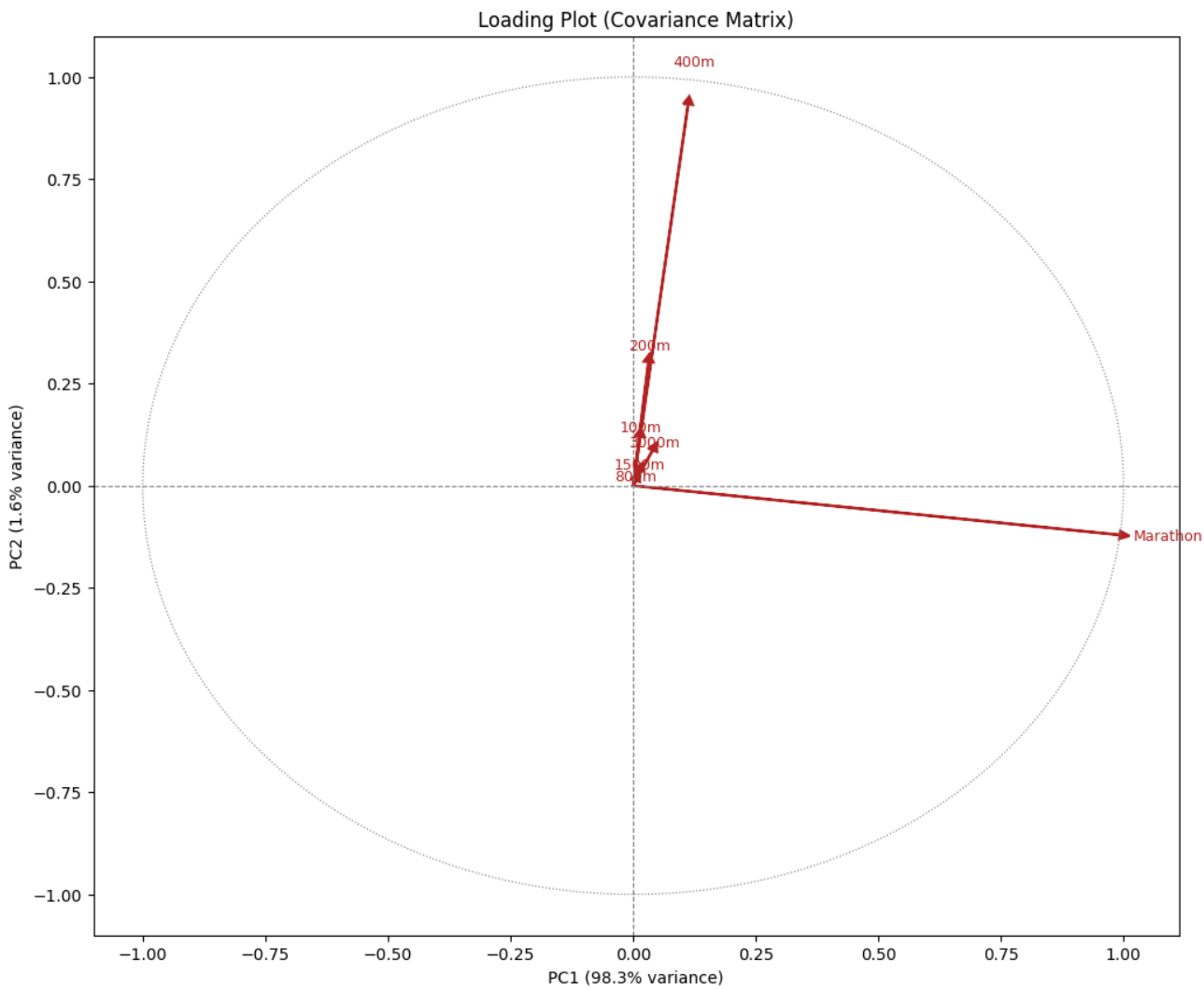
In [35]:
```
pca_full_cov = PCA(n_components=7)
pca_full_cov.fit(X_scaled_cov)

eigenvalues = pca_full_cov.explained_variance_  # actual eigenvalues
ev = pca_full_cov.explained_variance_ratio_
cumev = np.cumsum(ev)
n = len(eigenvalues)
x = np.arange(1, n + 1)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))

# --- Scree Plot ---
ax1.plot(x, eigenvalues, 'o-', color='steelblue', linewidth=2, markersize=7)
ax1.axvline(x=2, color='red', linestyle='--', linewidth=1.5, label='m = 2')
ax1.set_xlabel('Principal component index')
ax1.set_ylabel('Eigenvalue')
ax1.set_title('Scree plot')
ax1.set_xticks(x)
ax1.legend()

# --- Cumulative PVE Plot ---
ax2.plot(x, cumev, 'o-', color='darkgreen', linewidth=2, markersize=7)
```

```
# Threshold lines
ax2.axhline(y=0.80, color='gray', linestyle=':', linewidth=1, label='80%')
ax2.axhline(y=0.90, color='red', linestyle='--', linewidth=1, label='90%')
ax2.axhline(y=0.95, color='gray', linestyle='--', linewidth=1, label='95%')

# Selected m vertical line
selected_m = 2
ax2.axvline(x=selected_m, color='red', linestyle='--', linewidth=1.5)

# Annotate the selected point
ax2.annotate(f'(m={selected_m}), cumPVE={cumev[selected_m-1]:.3f}',
             xy=(selected_m, cumev[selected_m-1]),
             xytext=(selected_m + 0.5, cumev[selected_m-1] - 0.08),
             fontsize=9, color='darkgreen',
             arrowprops=dict(arrowstyle='->', color='darkgreen'))
ax2.plot(selected_m, cumev[selected_m-1], 'o', color='gold', markersize=10,

ax2.set_xlabel('Principal component index')
ax2.set_ylabel('Cumulative proportion of variance explained')
ax2.set_title('Cumulative PVE')
ax2.set_xticks(x)
ax2.set_ylim(0, 1.05)
ax2.yaxis.set_major_formatter(mtick.FuncFormatter(lambda v, _: f'{v:.1f}'))
ax2.legend(loc='lower right', fontsize=8)

plt.tight_layout()
plt.show()
```



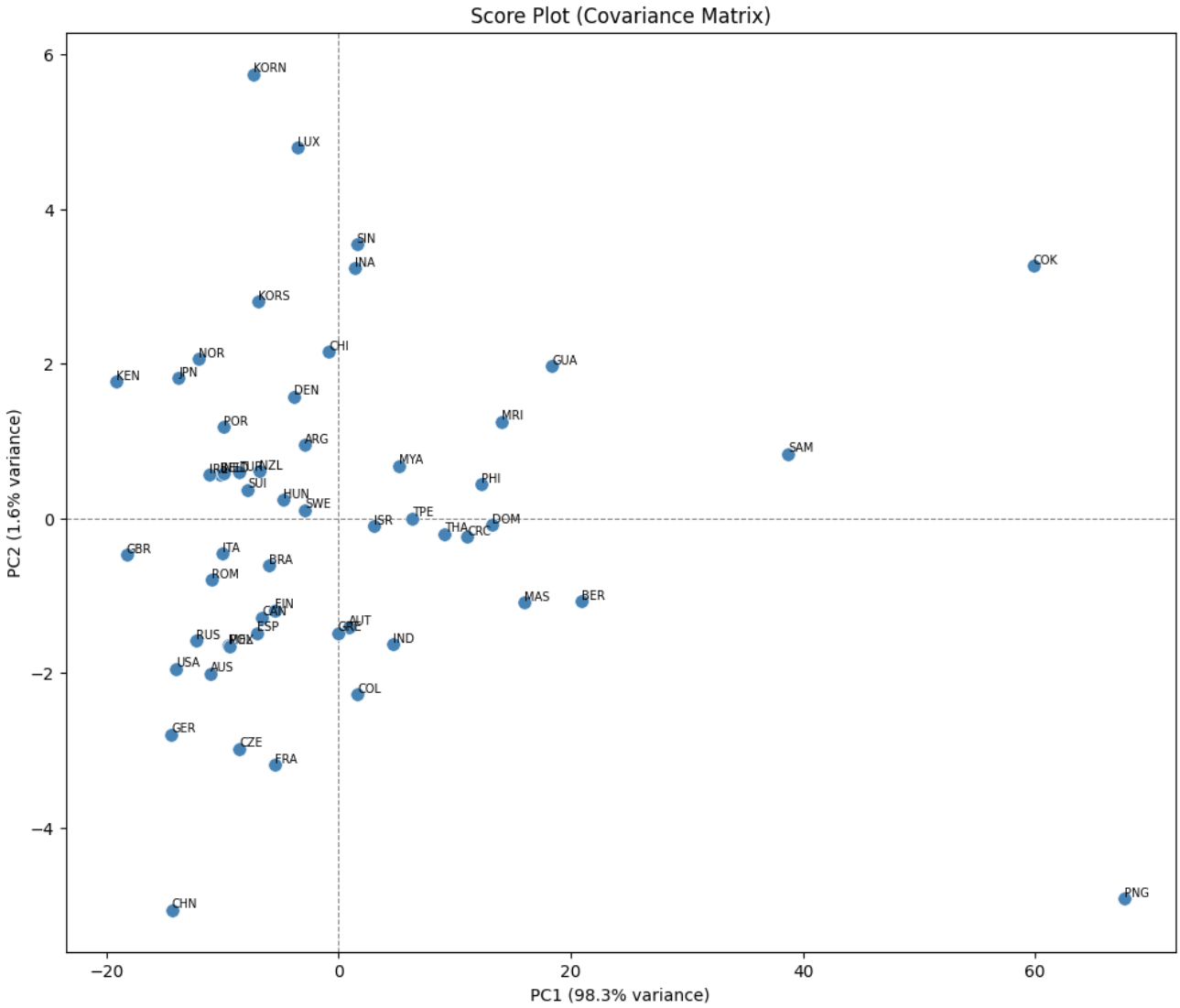
In [36]:
```
scores_cov = principal_components_cov  # from your original pca.fit_transfor
loadings_cov = pca_cov.components_.T   # shape (7, 2)
labels = numerical_df.columns

plt.figure(figsize=(12, 10))
for i, var in enumerate(labels):
    plt.arrow(0, 0, loadings_cov[i, 0], loadings_cov[i, 1],
```

```
                head_width=0.02, head_length=0.02,
                fc='firebrick', ec='firebrick', linewidth=1.5)
    plt.annotate(var, (loadings_cov[i, 0]*1.1, loadings_cov[i, 1]*1.1),
                 fontsize=9, color='firebrick', ha='center')
plt.axhline(0, color='gray', linewidth=0.8, linestyle='--')
plt.axvline(0, color='gray', linewidth=0.8, linestyle='--')
theta = np.linspace(0, 2*np.pi, 200)
plt.plot(np.cos(theta), np.sin(theta), color='gray', linewidth=0.8, linestyl
plt.xlabel(f'PC1 ({pca_cov.explained_variance_ratio_[0]*100:.1f}% variance)'
plt.ylabel(f'PC2 ({pca_cov.explained_variance_ratio_[1]*100:.1f}% variance)'
plt.title('Loading Plot (Covariance Matrix)')
plt.show()
```



In [37]:
```
plt.figure (figsize=(12, 10))
plt.scatter(scores_cov[:, 0], scores_cov[:, 1], color='steelblue', edgecolor
            linewidth=0.5, s=70, zorder=3)
for i, country in enumerate(df['Country']):
    plt.annotate(country, (scores_cov[i, 0], scores_cov[i, 1]),
                 fontsize=7, ha='left', va='bottom')
plt.axhline(0, color='gray', linewidth=0.8, linestyle='--')
```

```
plt.axvline(0, color='gray', linewidth=0.8, linestyle='--')
plt.xlabel(f'PC1 ({pca_cov.explained_variance_ratio_[0]*100:.1f}% variance)'
plt.ylabel(f'PC2 ({pca_cov.explained_variance_ratio_[1]*100:.1f}% variance)'
plt.title('Score Plot (Covariance Matrix)')
plt.show()
```



- Since the original data has different metrics (seconds vs minutes vs others), the column with the largest units (Marathon) dominated the PCA, with the first PC1 capturing 98.3% of the variance. The loadings plot shows that Marathon times variable strongly positvely correlates with the PC1, which makes sense since Marathon times has the most covariance. Compared to the correlation PCA, the Scree plot demonstrates that the first principal component (Marathon) captured over 95% of the variance. And compared to the correlation plot, the covariance PCA failed to capture any distinction between a country's specilization in long distance and short distance running.

g.

- Covariance matrix PCA is sensitive to the scale of variables, specifically when variables are measured in different units or have very different magnitudes, high-variance variables will dominate the principal components, as seen with Marathon times here. In practice, covariance PCA is appropriate when all variables are measured on the same scale and their relative variances are meaningful (e.g. stock returns in the same units, where you want larger-variance assets to contribute more). Correlation matrix PCA is preferred when variables are on different scales or when equal weighting of all variables is desired regardless of their raw variance. For this running dataset, correlation PCA is clearly more appropriate as it reveals the meaningful sprint vs endurance structure that covariance PCA obscures

Problem 4: Yield Curve PCA and Reconstruction

In [43]:
```
treasury_df = pd.read_csv('treasury_yield_panel.csv')
treasury_df.set_index('date', inplace=True)
treasury_df.head()
```

Out[43]:

	X1Y	X2Y	X3Y	X5Y	X7Y	X10Y	X20Y	X30Y
date								
2014-01-02	0.13	0.39	0.76	1.72	2.41	3.00	3.68	3.92
2014-01-03	0.13	0.41	0.80	1.73	2.42	3.01	3.69	3.93
2014-01-06	0.12	0.40	0.78	1.70	2.38	2.98	3.66	3.90
2014-01-07	0.13	0.40	0.80	1.69	2.37	2.96	3.64	3.88
2014-01-08	0.13	0.43	0.87	1.77	2.44	3.01	3.67	3.90

a.

In [45]:
```
treasury_df = treasury_df.diff().dropna()
treasury_df = treasury_df/ (0.01/100)
treasury_df.head()
```

Out[45]:

date	X1Y	X2Y	X3Y	X5Y	X7Y	X10Y	X20Y	X30Y
2014-01-06	-100.0	-300.0	-600.0	-400.0	-500.0	-400.0	-400.0	-400.0
2014-01-07	200.0	100.0	400.0	200.0	300.0	100.0	100.0	100.0
2014-01-08	-100.0	300.0	500.0	900.0	800.0	700.0	500.0	400.0
2014-01-09	0.0	-200.0	-800.0	-1000.0	-1000.0	-900.0	-800.0	-400.0
2014-01-10	-100.0	-600.0	-800.0	-900.0	-900.0	-500.0	-300.0	-600.0

b.

In [47]:

```
treasury_cov = scaler_cov.fit_transform(treasury_df)
treasury_pca = PCA(n_components=3)
treasury_pca.fit(treasury_cov)
loadings_treasury = pd.DataFrame(
    treasury_pca.components_.T,
    index=treasury_df.columns,
    columns=['PC1', 'PC2', 'PC3']
)

display(loadings_treasury)
print("Explained variance ratios:", treasury_pca.explained_variance_ratio_)
```

	PC1	PC2	PC3
X1Y	0.199641	-0.427425	0.792473
X2Y	0.335361	-0.467799	0.005785
X3Y	0.373057	-0.342419	-0.245312
X5Y	0.403059	-0.118336	-0.326920
X7Y	0.410275	0.059021	-0.234252
X10Y	0.383061	0.221597	-0.054599
X20Y	0.349283	0.432434	0.216895
X30Y	0.330134	0.477106	0.316229

```
Explained variance ratios: [0.84361947 0.11181427 0.02195435]
```

- The varaince explained by PC1 is 84.4%, PC2 is 11.2%, PC3 is 2.2%

c. Rank 2 PCA reconstruction

i.

Since r_i is `p x 1`, v_1 and v_2 are `p x 1`

$$\text{if AB} = 0\text{, then A and B are orthogonal}$$

$$\begin{aligned}
&v_1^T r_i = v_1^T(x_{c,i} - \hat{x_{c,i}}) \\
&= v_1^T(x_{c,i}) - v_1^T(\hat{x_{c,i}}) \\
&= z_{i1} - v_1^T(\hat{x_{c,i}}) \\
&= z_{i1} - v_1^T(z_{i1}v_1 + z_{i2}v_2)
\end{aligned}$$

$$\text{Since } v_1 \text{ and } v_2 \text{ are orthonormal, } v_1^T \cdot v_2 = 0$$

$$= z_{i1} - z_{i1}v_1^T v_1 + 0 = 0$$

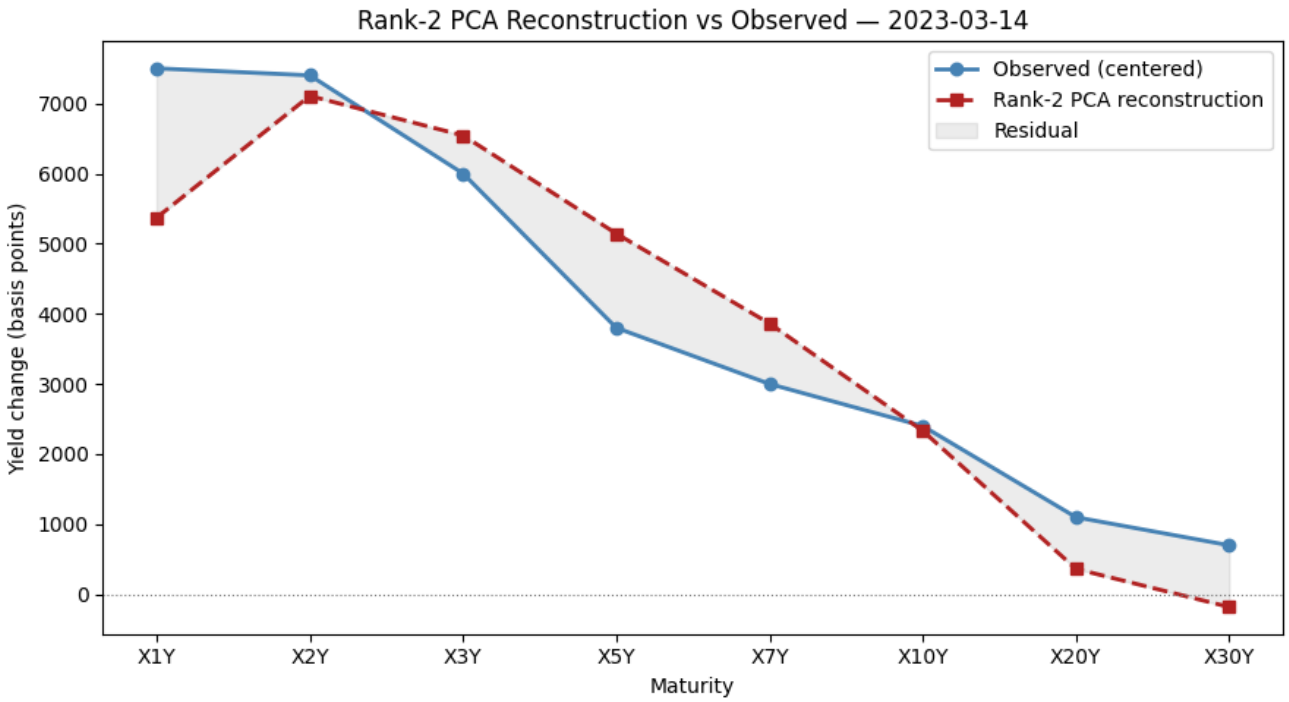
$$\begin{aligned}
&\text{Similarly, for } v_2 \\
&v_2^T r_i = v_2^T(x_{c,i} - \hat{x_{c,i}}) \\
&= z_{i2} - v_2^T(\hat{x_{c,i}}) \\
&= z_{i2} - 0 - v_2^T z_{i2} v_2 = 0 \\
&\text{Therefore, } r_i \text{ is orthogonal to both } v_1 \text{ and } v_2
\end{aligned}$$

ii.

In [56]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import numpy as np


scaler = StandardScaler(with_std=False)
X_centered = scaler.fit_transform(treasury_df)


pca = PCA(n_components=8)
scores = pca.fit_transform(X_centered)
loadings = pca.components_


pc3_scores = scores[:, 2]
worst_day_idx = np.argmax(np.abs(pc3_scores))
```

```
worst_day = treasury_df.index[worst_day_idx]
print(f"Selected day: {worst_day}, |z_i3| = {np.abs(pc3_scores[worst_day_idx


x_obs = X_centered[worst_day_idx]


v1, v2 = loadings[0], loadings[1]
z_i1, z_i2 = scores[worst_day_idx, 0], scores[worst_day_idx, 1]
x_hat = z_i1 * v1 + z_i2 * v2


residual = x_obs - x_hat
print(f"Residual norm: {np.linalg.norm(residual):.4f}")


maturities = treasury_df.columns.tolist()

plt.figure(figsize=(9, 5))
plt.plot(maturities, x_obs, 'o-', color='steelblue', linewidth=2,
         markersize=6, label='Observed (centered)')
plt.plot(maturities, x_hat, 's--', color='firebrick', linewidth=2,
         markersize=6, label='Rank-2 PCA reconstruction')
plt.fill_between(range(len(maturities)), x_obs, x_hat,
                 alpha=0.15, color='gray', label='Residual')

plt.xticks(range(len(maturities)), maturities)
plt.axhline(0, color='gray', linewidth=0.8, linestyle=':')
plt.xlabel('Maturity')
plt.ylabel('Yield change (basis points)')
plt.title(f'Rank-2 PCA Reconstruction vs Observed — {worst_day}')
plt.legend()
plt.tight_layout()
plt.show()
```

```
Selected day: 2023-03-14, |z_i3| = 2898.31
Residual norm: 2964.4407
```



- The day with the largest $|z_{i3}|$ represents a day where the rank-2 approximation is most visibly imperfect, as shown in the plot above.
- The aspect of the daily yield curve movement not captured by the first two PCs is the curvature of the yield curve. PC1 captures parallel level shifts , and PC2 captures slope changes. However, PC3 corresponds to curvature. On the selected day, the residual r_i is non-trivial precisely because the yield curve exhibited this curvature pattern, which cannot be reconstructed from v_1 and v_2 alone.

Problem 5. Factor Analysis

In [ ]:
```
from sklearn.preprocessing import StandardScaler
from numpy.linalg import eig


X = numerical_df.values
X_std = (X - X.mean(axis=0)) / X.std(axis=0)

# Factor analysis via PCA on correlation matrix (principal factor method)
R = np.corrcoef(X_std.T)
eigenvalues, eigenvectors = eig(R)

# Sort by descending eigenvalue
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx].real
```

```
eigenvectors = eigenvectors[:, idx].real

# Extract 2-factor loadings: L = V * sqrt(Lambda)
n_factors = 2
L = eigenvectors[:, :n_factors] * np.sqrt(eigenvalues[:n_factors])

loadings_fa = pd.DataFrame(
    L,
    index=numerical_df.columns,
    columns=['Factor1', 'Factor2']
)

# Uniquenesses = 1 - communalities
communalities = np.sum(L**2, axis=1)
uniquenesses = pd.Series(1 - communalities, index=numerical_df.columns, name

print("=== Estimated Loading Matrix ===")
print(loadings_fa.round(4))
print("\n=== Uniquenesses ===")
print(uniquenesses.round(4))
```

```
=== Estimated Loading Matrix ===
          Factor1  Factor2
100m      -0.8883   0.3927
200m      -0.8927   0.4122
400m      -0.8949   0.2846
800m      -0.9429  -0.1898
1500m     -0.9447  -0.1979
3000m     -0.9155  -0.2730
Marathon  -0.8334  -0.4267

=== Uniquenesses ===
100m        0.0566
200m        0.0331
400m        0.1182
800m        0.0748
1500m       0.0683
3000m       0.0873
Marathon    0.1234
Name: Uniqueness, dtype: float64
```

b.

- Based on the results above, 800m, 1500m appear to load the most negatively strong on factor 1, although for factor 1, all 7 events load strongly against it. It seems that factor 1 is similar to the correlation PCA where this captures the overall athletic ability of a nation. For factor 2, the long distance events load negatively against the factor, with Marathon being the strongest, and the short distance events load

positively (with 200m being the most positive).

- In terms of uniqueness, the Marathon event has the largest uniquness, suggesting that it is more difficult of being captured by the two factors.

C.

- Similarity: Both FA and PCA identify the same two latent dimensions, an overall speed factor and a distance factor. And the sign patterns of the loadings are consistent across both methods.
- Difference: PCA treats all variance as signal and fully reconstructs each variable from the components. Factor analysis explicitly decomposes variance into shared variance and variable-specific variance. The low uniquenesses here suggest the running events are almost entirely driven by common factors, meaning both methods reach similar conclusions for this dataset.